

Penetration Testing Report

Mediroza General Hospital

Batch B083 | Week 4 | Black-Box Penetration Test

Field	Details
Client	Mediroza General Hospital
Target	https://medirozahospital.com
Project Type	Penetration Testing & Vulnerability Assessment
Duration	5 Days
Prepared By	Adebayo Judith Tosin
Report Date	29-09-26
Confidentiality	Confidential — Authorized Personnel Only

This report documents a black-box penetration test conducted under written authorization from the client, in a controlled educational environment. Findings, techniques, and recommendations herein apply only to the authorized target and must not be replicated against systems without explicit permission.

1. Executive Summary

Overview

On the 29 September 2026, NetworkWalks conducted an authorized black-box penetration test against Mediroza General Hospital's public-facing web application (<https://medirozahospital.com>), as part of Batch B083, Week 4. Testing was scoped to the target domain only, under written authorization covering all reconnaissance, exploitation, and data-analysis activities described in this report.

Key Findings

- A robots.txt file inadvertently revealed the existence of sensitive, undocumented directories (/patient/, /staff/, /old/), which directly led to the discovery of the two Critical findings below.
- An unauthenticated, publicly downloadable database backup (/old/mediroza_db_backup_2019.sql) exposed the personal and financial details of all 30 hospital staff members — including national ID numbers, phone numbers, and salaries — plus confidential shareholder equity data.
- A SQL injection vulnerability in the Patient Portal login form (/patient/login.php) allowed complete authentication bypass, granting unauthenticated access to confidential patient lab reports.
- The three retrieved lab report PDFs were protected with weak, dictionary-crackable passwords, offering minimal real protection once the files were exfiltrated.

Overall Risk to the Client

Taken together, these findings represent a Critical overall risk to Mediroza General Hospital. An attacker with no credentials and no insider knowledge could, using only a web browser, obtain confidential patient lab results, full staff personal and payroll records, and confidential shareholder information. This exposes the hospital to regulatory and patient-data-protection compliance risk, identity theft and phishing risk for staff, financial harm, and reputational damage. Remediation of the two Critical findings (Findings 2 and 3) should be treated as an immediate priority, followed by the remaining findings in this report.

2. Scope and Methodology

Target

<https://medirozahospital.com> (resolved IP: 199.188.201.16)

Project Details

Field	Details
Project Type	Penetration Testing & Vulnerability Assessment
Scope	Full black-box penetration test of the target domain only
Rules of Engagement	Testing limited to the target domain. No social engineering. No denial of service. No testing outside agreed scope.
Authorization	Written authorization granted by the client for all milestones (M1-M4)

Tools Used

Tool	Purpose	Notes
whois	Domain registration recon	Confirmed privacy-protected registration via Namecheap; no useful registrant PII exposed here.
nslookup / dnsrecon	DNS resolution & records	Resolved medirozahospital.com to 199.188.201.16.
whatweb	Web server / tech stack fingerprinting	Identified LiteSpeed server; requests returned 403, later found to be WAF filtering of scanner-like traffic rather than a real block.
theHarvester	Subdomain / email enumeration	Found ftp.medirozahospital.com and mail.medirozahospital.com, plus contact email info@medirozahospital.com.
wafw00f	WAF detection	Confirmed the site sits behind a LiteSpeed WAF, explaining earlier blocked scanner requests.
Firefox (manual browsing)	Manual site mapping, robots.txt, sitemap.xml review	robots.txt revealed /patient/, /staff/, and /old/, which led directly to Findings 1-3.

NetworkWalks password cracker tool	Offline PDF password cracking (dictionary attack)	Cracked all 3 PDF passwords; see Finding 4 for wordlists and timings per file.
------------------------------------	---	--

Approach

Testing followed a standard black-box methodology: passive reconnaissance (whois, DNS, subdomain and email enumeration) to map the target's footprint; manual enumeration of the live site, including robots.txt and sitemap.xml, to identify hidden or undocumented paths; exploitation of the vulnerabilities this enumeration surfaced (directory listing, SQL injection); analysis of the data recovered from each exploited vulnerability; and finally consolidation of all findings into this report.

Limitations

The target's WAF blocked or altered responses to some automated tool requests (e.g. whatweb, direct curl requests to error_log), which meant some steps had to be repeated through a standard browser to get a true picture of the application. FTP access to ftp.medirozahospital.com was identified but not tested further, as no valid credentials were found during this engagement. Testing was scoped to the target domain only and completed within the 5-day project window; a deeper, unscoped assessment may surface additional issues not covered here.

3. Findings and Proof of Exploitation

Each vulnerability is documented below with a description, reproduction steps, supporting evidence, and impact. Add or remove findings as needed to match what you actually tested and confirmed.

Finding 1: Sensitive Paths Disclosed via robots.txt

Suggested Risk Rating: Low / Informational (confirm this in Section 4)

Description

During initial reconnaissance, robots.txt was requested directly at <https://medirozahospital.com/robots.txt>. The file explicitly lists three directories — /patient/, /staff/, and /old/ — that the site owner wanted excluded from search engine indexing. A Disallow directive only asks well-behaved search engine crawlers not to index a path; it provides no actual access control, so each directory could still be browsed directly by anyone who reads the file.

Steps to Reproduce

1. Navigate to <https://medirozahospital.com/robots.txt> in a browser. 2. Note the Disallow entries for /patient/, /staff/, and /old/. 3. Visit each disallowed path directly (e.g. <https://medirozahospital.com/old/>) and confirm it is reachable.

Evidence

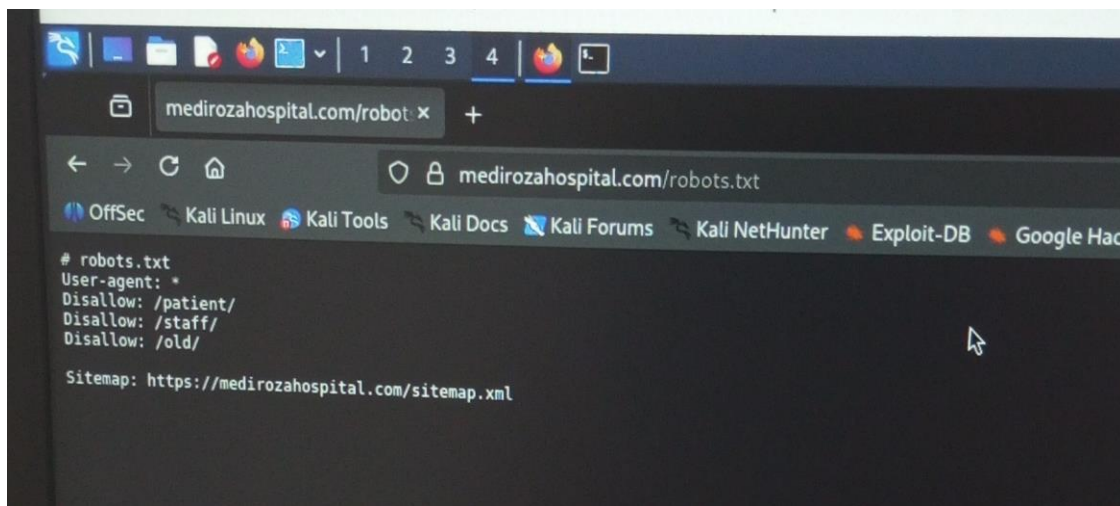


Figure 1: robots.txt discloses the /patient/, /staff/, and /old/ paths.

Impact

On its own this is low severity, but it materially reduced the effort needed to find the two Critical findings below, by pointing an attacker directly at the interesting parts of the application. Sensitive or administrative paths should never rely on being “hidden” via robots.txt.

Finding 2: Publicly Accessible Historic Database Backup (/old/)

Suggested Risk Rating: Critical (confirm this in Section 4)

Description

The /old/ directory identified above had directory listing enabled and contained a single file, mediroza_db_backup_2019.sql — a complete, unauthenticated SQL database backup of the hospital's

internal “mediroza_hr” database (per the file's own header, generated by “Mediroza CMS 1.4.2 backup module” and explicitly commented as containing confidential staff and shareholder records). The file was downloadable by anyone, with no authentication required.

Steps to Reproduce

1. Browse to <https://medirozahospital.com/old/>. 2. Observe the directory listing exposing mediroza_db_backup_2019.sql. 3. Download the file directly, e.g.: `wget https://medirozahospital.com/old/mediroza_db_backup_2019.sql` 4. Open the file in a text editor and review its contents.

Evidence

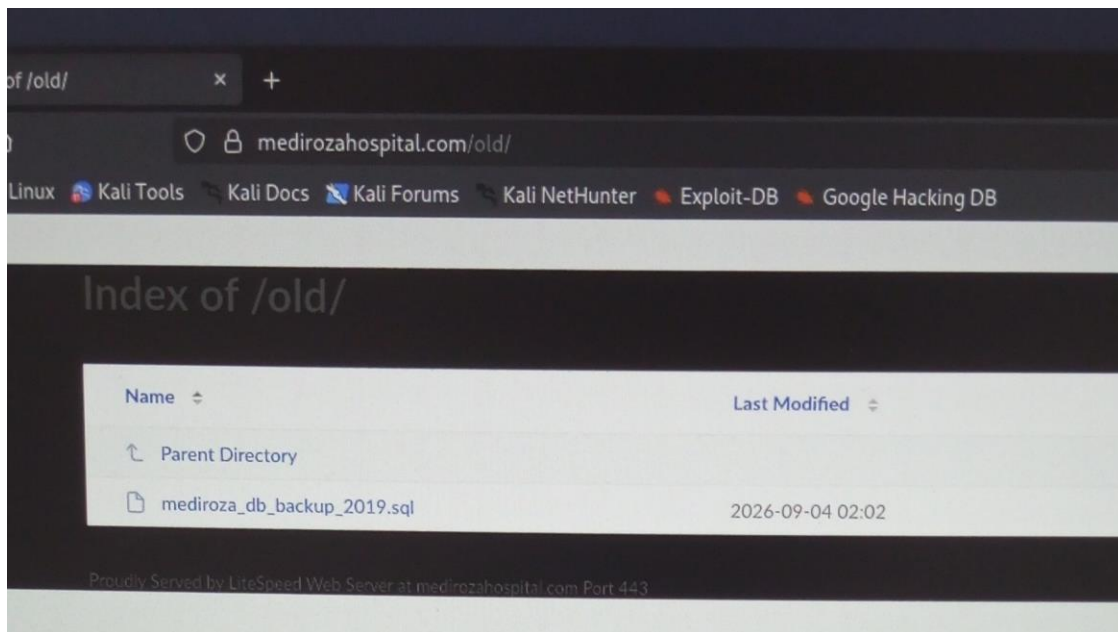


Figure 2: Directory listing for /old/ exposing the backup file.

```
medirozahospital.com/old/m +
medirozahospital.com/old/mediroza_db_backup_2019.sql
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

-- mediroza.db backup 2019.sql
-- Mediroza General Hospital - internal database backup
-- Generated by Mediroza CMS 1.4.2 backup module
-- Host: localhost Database: mediroza_hr
-- WARNING: contains confidential staff and shareholder records
-- Backup date: 2019-08-27 02:14:03

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

--
-- Table structure for table 'staff'
--
DROP TABLE IF EXISTS `staff`;
CREATE TABLE `staff` (
  `id` int(11) NOT NULL AUTO INCREMENT,
  `full name` varchar(120) NOT NULL,
  `job title` varchar(120) NOT NULL,
  `department` varchar(80) NOT NULL,
  `email` varchar(120) NOT NULL,
  `phone` varchar(20) NOT NULL,
  `national_id` varchar(20) NOT NULL,
  `monthly salary zar` int(11) NOT NULL,
  `date joined` date NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--
-- Dumping data for table 'staff'
--
LOCK TABLES `staff` WRITE;
INSERT INTO `staff` (`id`,`full name`,`job title`,`department`,`email`,`phone`,`national_id`,`monthly salary zar`,`date joined`) VALUES
(1,'Dr. Rajesh Naidoo','Chief Pathologist','Diagnostics Lab','r.naidoo@medirozahospital.com','+27 82 101 2007','850210130110081',138000,'2009-03-16'),
(2,'Sarah Botha','Chief Financial Officer','Finance','s.botha@medirozahospital.com','+27 82 102 2014','850310230220082',152000,'2011-07-01'),
(3,'Dr. Johan van der Merwe','Medical Director','Management','j.merwe@medirozahospital.com','+27 82 103 2021','850410330330083',160000,'2007-01-22'),
(4,'Dr. Anita Naicker','Consultant Cardiologist','Cardiology','a.naicker@medirozahospital.com','+27 82 104 2028','850510430440084',132000,'2012-09-10'),
(5,'Dr. Ahmed Kara','Consultant Physician','Internal Medicine','a.kara@medirozahospital.com','+27 82 105 2035','850610530550085',128000,'2013-02-18'),
(6,'Dr. Yusuf Cassin','Senior Registrar','Emergency & Trauma','y.cassin@medirozahospital.com','+27 82 106 2042','850710630660086',74000,'2018-05-04'),
(7,'Michael Roberts','HR Director','Human Resources','m.roberts@medirozahospital.com','+27 82 107 2049','850810730770087',96000,'2010-11-15'),
(8,'Susan Pretorius','HR Officer','Human Resources','s.pretorius@medirozahospital.com','+27 82 108 2056','850910830880088',32000,'2016-08-23'),
(9,'Jameel Malik','IT Systems Administrator','IT','j.malik@medirozahospital.com','+27 82 109 2063','851010930990089',58000,'2015-04-12'),
(10,'Thabo Molefe','Network Engineer','IT','t.molefe@medirozahospital.com','+27 82 110 2070','851111031100081',46000,'2017-10-02'),
(11,'Nomvula Khumalo','Registered Nurse','Emergency & Trauma','n.khumalo@medirozahospital.com','+27 82 111 2077','851211131210082',34000,'2016-01-19'),
(12,'Lerato Mokoena','Registered Nurse','Pediatrics','l.mokoena@medirozahospital.com','+27 82 112 2084','850111231220083',33600,'2017-06-07'),
(13,'Bongani Ndlovu','Registered Nurse','Cardiology','b.ndlovu@medirozahospital.com','+27 82 113 2091','850211331330084',35000,'2015-12-01'),
(14,'Zanele Mhlangu','Nursing Sister','Theatre','z.mhlangu@medirozahospital.com','+27 82 114 2098','850311431440085',42000,'2013-03-25'),
(15,'Kgabo Sithole','Pharmacist','Pharmacy','k.sithole@medirozahospital.com','+27 82 115 2105','850411531550086',61000,'2014-09-08'),
(16,'Naledi Zulu','Pharmacy Assistant','Pharmacy','n.zulu@medirozahospital.com','+27 82 116 2112','850511631660087',26000,'2019-02-14'),
```

Figure 3: Backup header confirming “Mediroza CMS 1.4.2” and the staff table, including national ID numbers and monthly salaries.

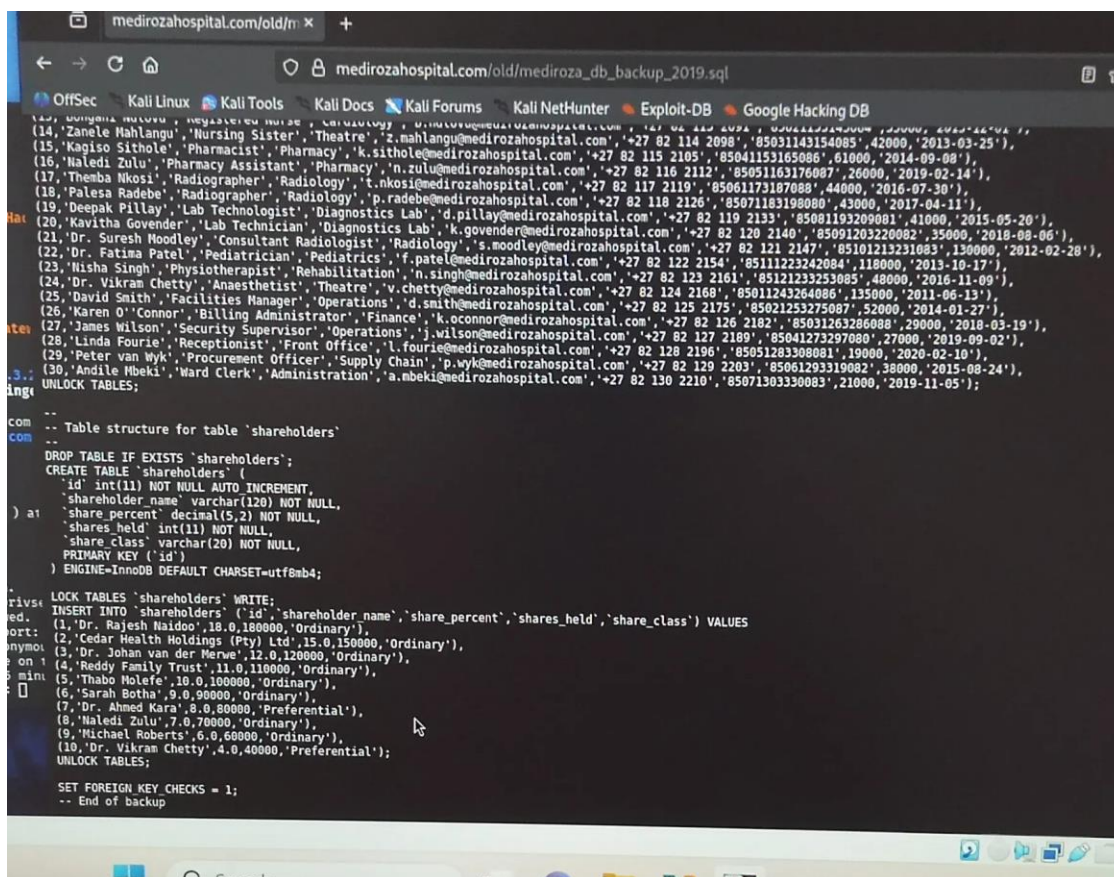


Figure 4: shareholders table with names, shareholding percentage, and shares held.

Impact

The backup contains the full staff table (30 records) with full name, job title, department, email, phone number, national ID number, and monthly salary for every employee — plus the shareholders table with names, shareholding percentage, shares held, and share class. This is a serious breach of staff PII (identity theft and targeted phishing risk from the exposed ID numbers, emails, and phone numbers) and of confidential financial/ownership information. Because the backup's own header flags it as confidential, this also points to a broader failure of data-handling practice: backups must never be stored inside a web-accessible directory.

Finding 3: SQL Injection — Authentication Bypass (Patient Portal)

Suggested Risk Rating: Critical (confirm this in Section 4)

Description

The Patient Portal login form at /patient/login.php submits a plain POST request with username and password fields, with no visible CSRF token (confirmed via View Page Source, Figure 5 below). Testing the login logic surfaced two issues. First, the application's error messages differ depending on whether the submitted username exists (“no username found” vs. “incorrect password”), allowing an attacker to enumerate valid accounts — this confirmed an admin account exists. Second, the username field is not sanitized before being used in the underlying SQL query: submitting admin'-- as the username, with any value in the password field, comments out the remainder of the SQL WHERE clause, causing the application to authenticate as the admin user without a valid password ever being checked.

1. Navigate to <https://medirozahospital.com/patient/login.php>.
2. In the Username field, enter: admin'--
3. Enter any value in the Password field and submit.
4. The application logs in as the admin user and displays the admin account's confidential lab reports, without a valid password ever being supplied.

[illegible]

Mediroza General Hospital — Penetration Testing Report | Page 9

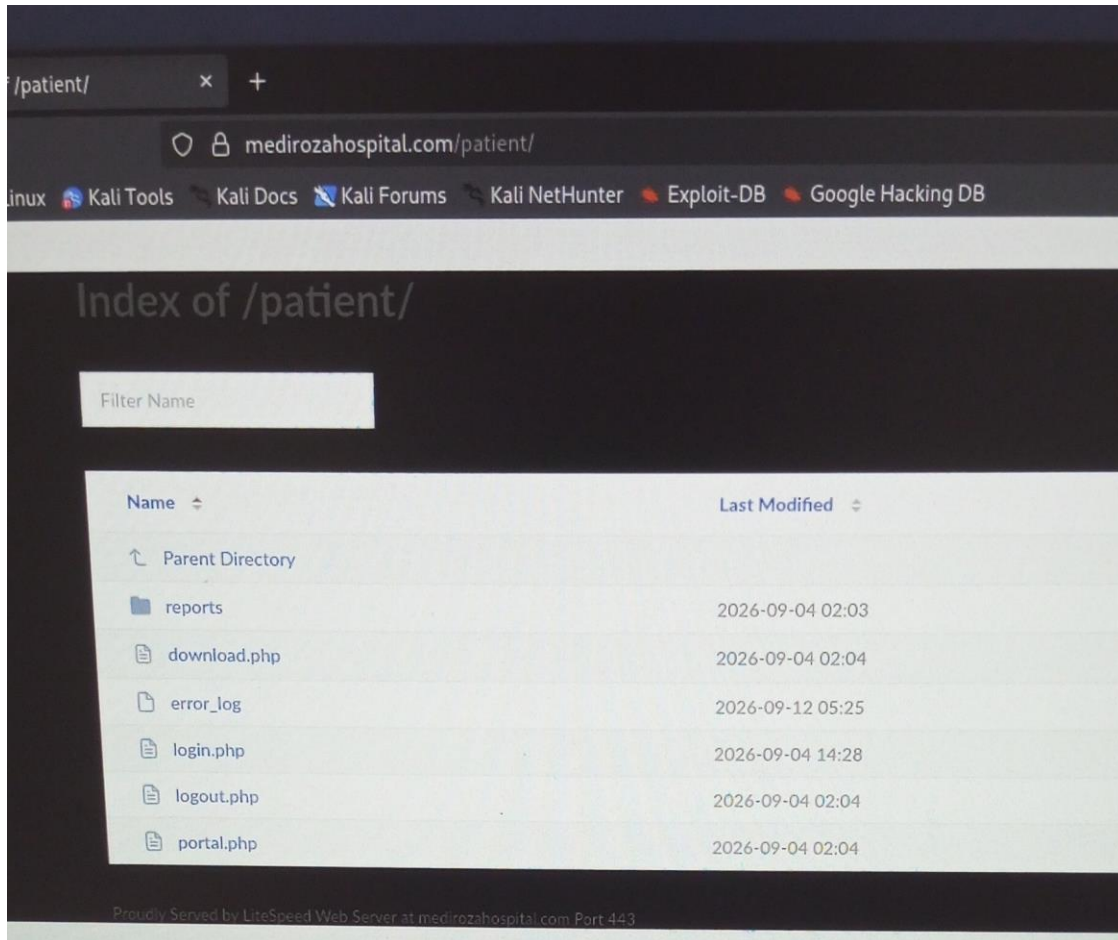


Figure 6: Directory listing of /patient/ showing login.php, portal.php, download.php, and the reports/ folder targeted by the bypass.

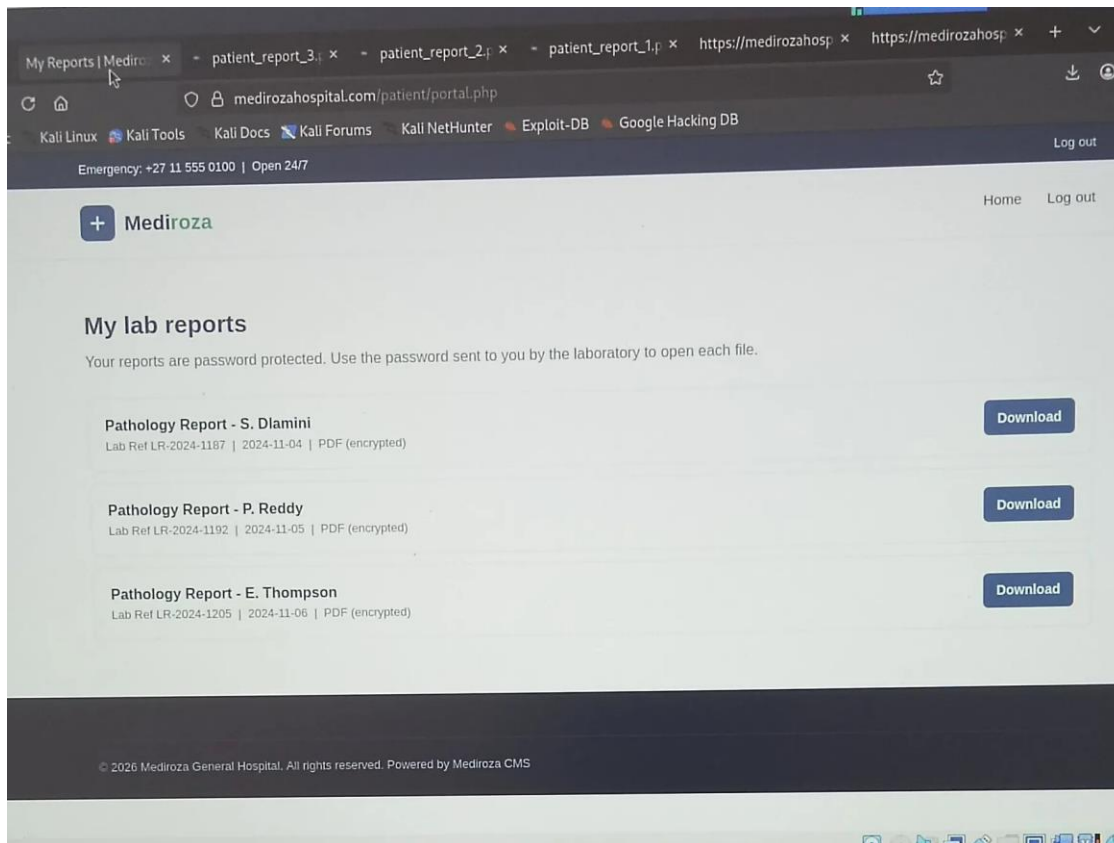


Figure 7: Successful admin'-- bypass — the Patient Portal “My lab reports” page, listing all 3 confidential pathology reports (S. Dlamini, P. Reddy, E. Thompson) with no valid password ever supplied.

Impact

This is a full authentication bypass via SQL injection, one of the most severe classes of web application vulnerability. It grants unauthenticated access to confidential patient lab reports and demonstrates that user input reaches a SQL query without parameterization — a pattern that likely affects other parts of the application built the same way.

Finding 4: Weak, Crackable Password Protection on Confidential PDF Reports

Suggested Risk Rating: High (confirm this in Section 4)

Description

The three lab report PDFs retrieved via the authentication bypass in Finding 3 (patient_report_1.pdf, patient_report_2.pdf, and patient_report_3.pdf) were password-protected using PDF R3 / 128-bit encryption. Each file's password was recovered using an offline dictionary attack (deriving keys with MD5 + RC4). patient_report_1.pdf and patient_report_2.pdf were both cracked almost instantly using a built-in 100-entry wordlist, recovering the passwords “123456” and “password” respectively — both extremely weak, commonly used passwords. patient_report_3.pdf used a stronger, non-dictionary password; a larger custom wordlist of 3,556 candidates was required, and the password “!@#\$\$%^&” was recovered after 3,546 attempts.

Steps to Reproduce

1. Download patient_report_1.pdf, patient_report_2.pdf, and patient_report_3.pdf from the Patient Portal (retrieved via Finding 3).
2. Submit each file's password hash to a PDF password cracking tool

configured for an offline dictionary attack. 3. Run the attack against each file in turn, starting with a small built-in wordlist before escalating to a larger custom wordlist. 4. Record the recovered password shown for each file.

Evidence

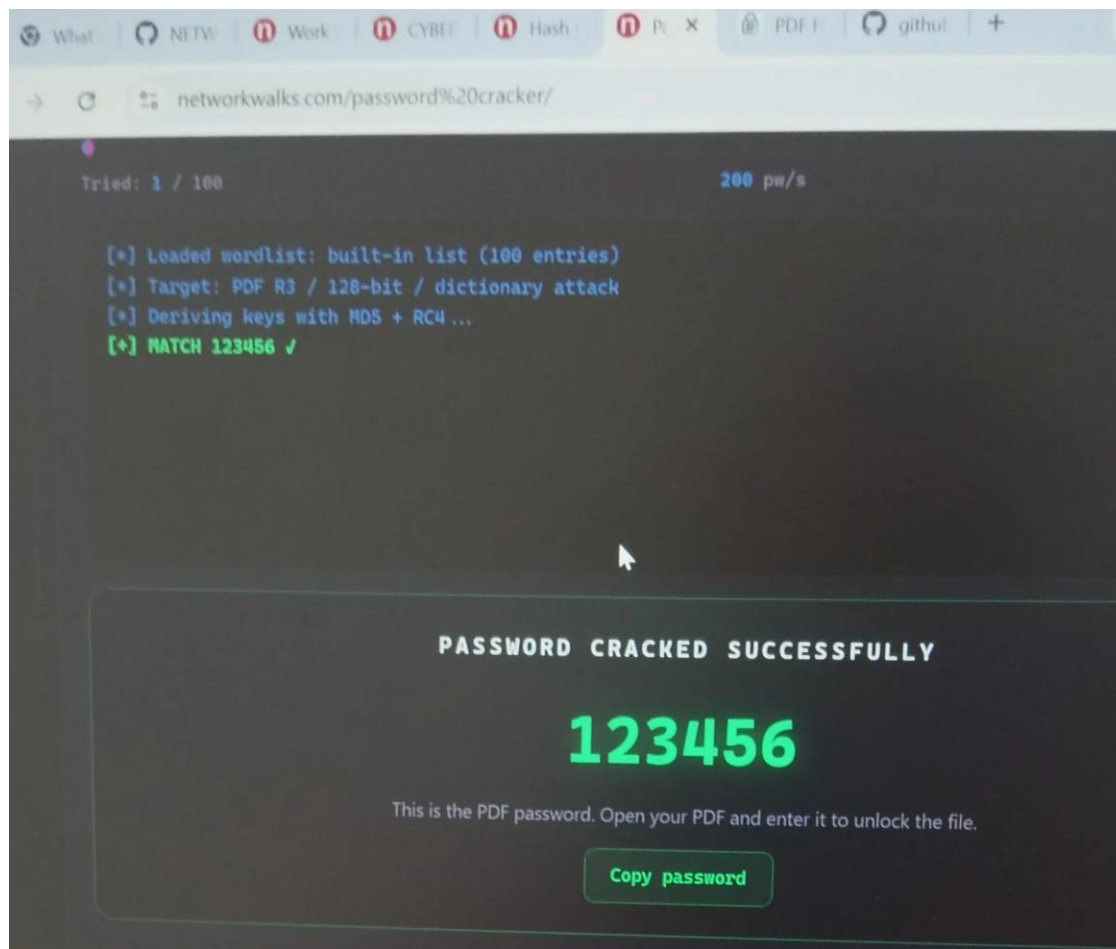


Figure 8: patient_report_1.pdf cracked instantly (1/100 attempts) — recovered password: 123456.

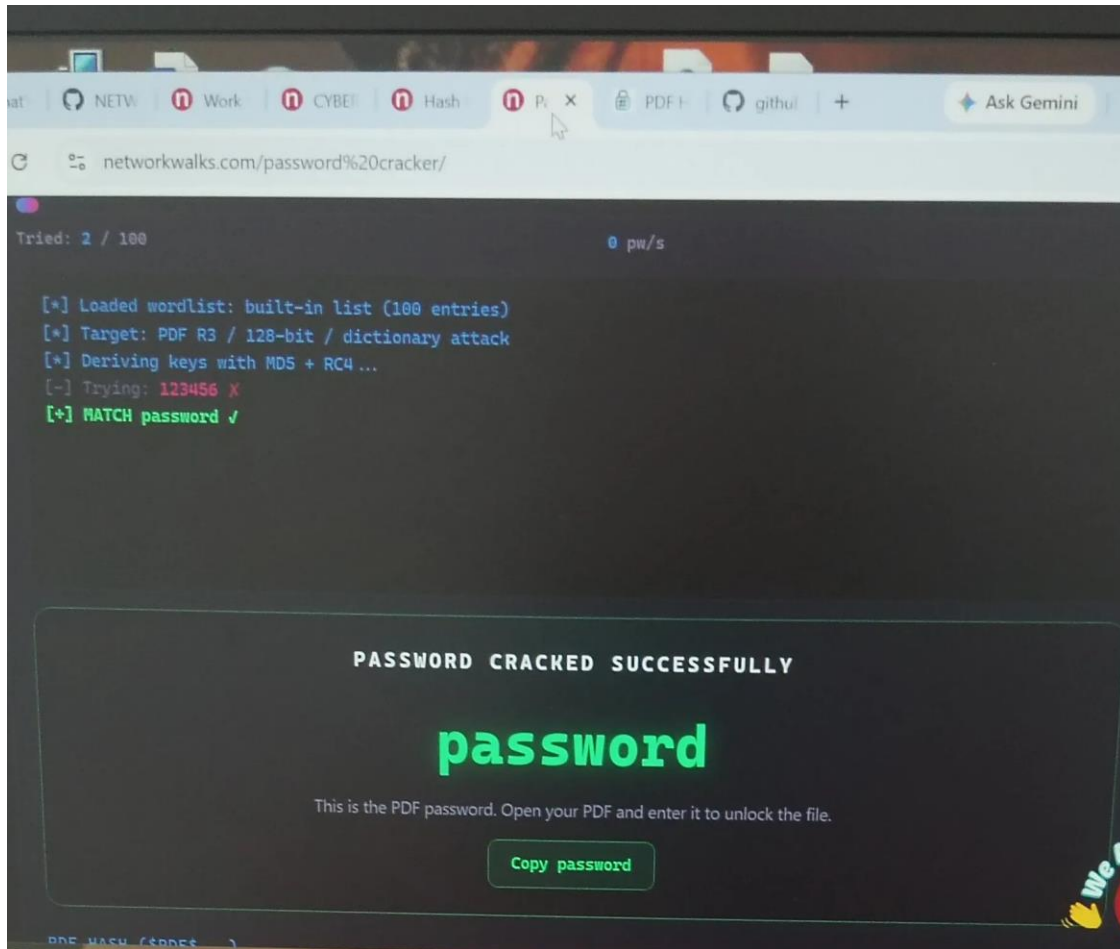


Figure 9: patient_report_2.pdf cracked on the 2nd attempt — recovered password: password.

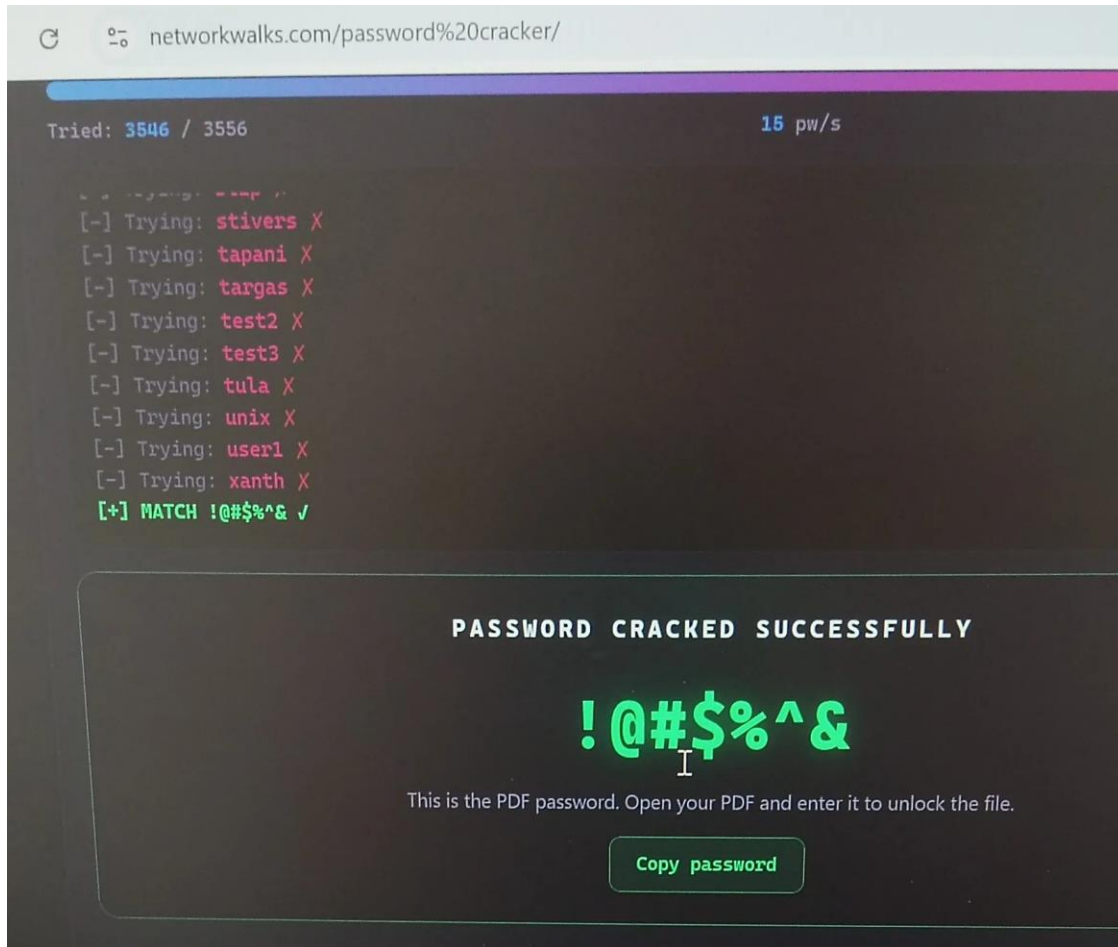


Figure 10: patient_report_3.pdf required a larger wordlist, cracked after 3,546 of 3,556 attempts — recovered password: !@#\$%^&.

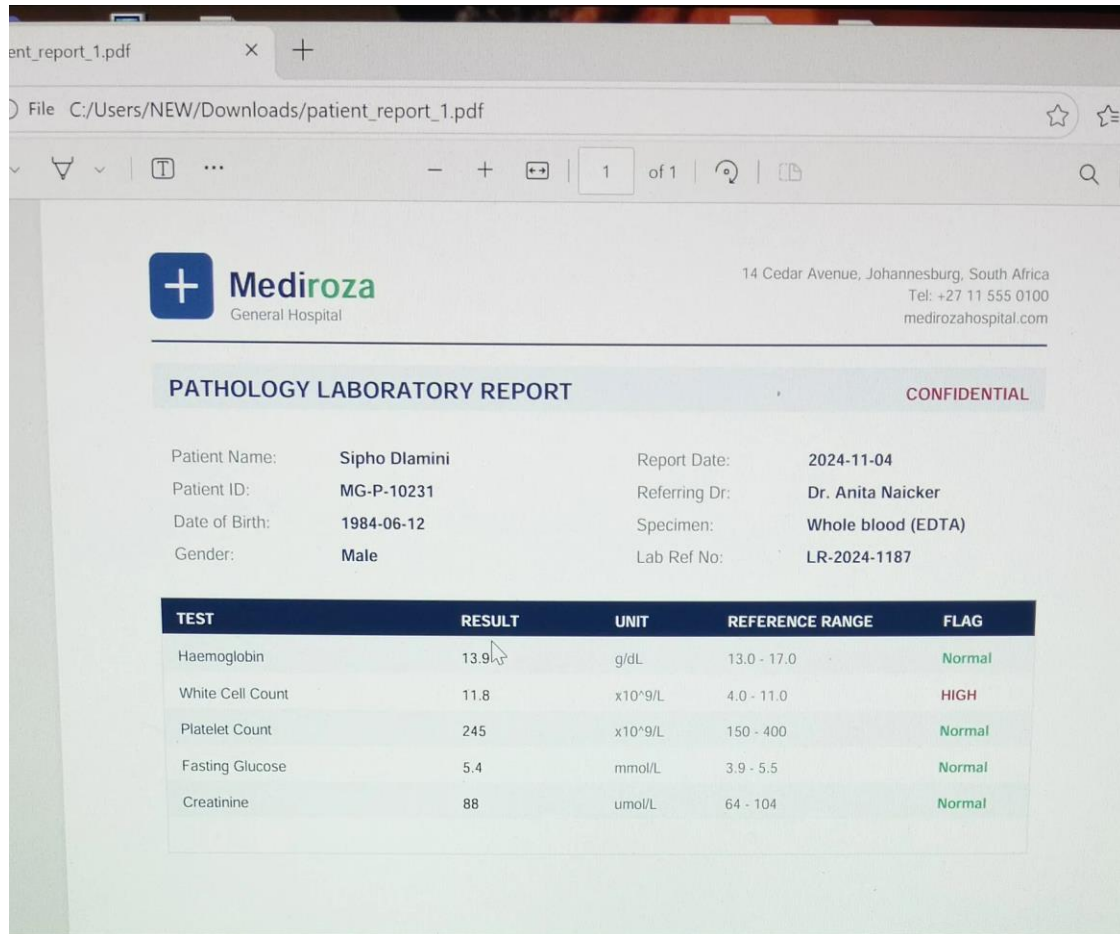


Figure 11: patient_report_1.pdf opened with password 123456, confirming genuine pathology report content for patient Siphso Dlamini (LR-2024-1187).

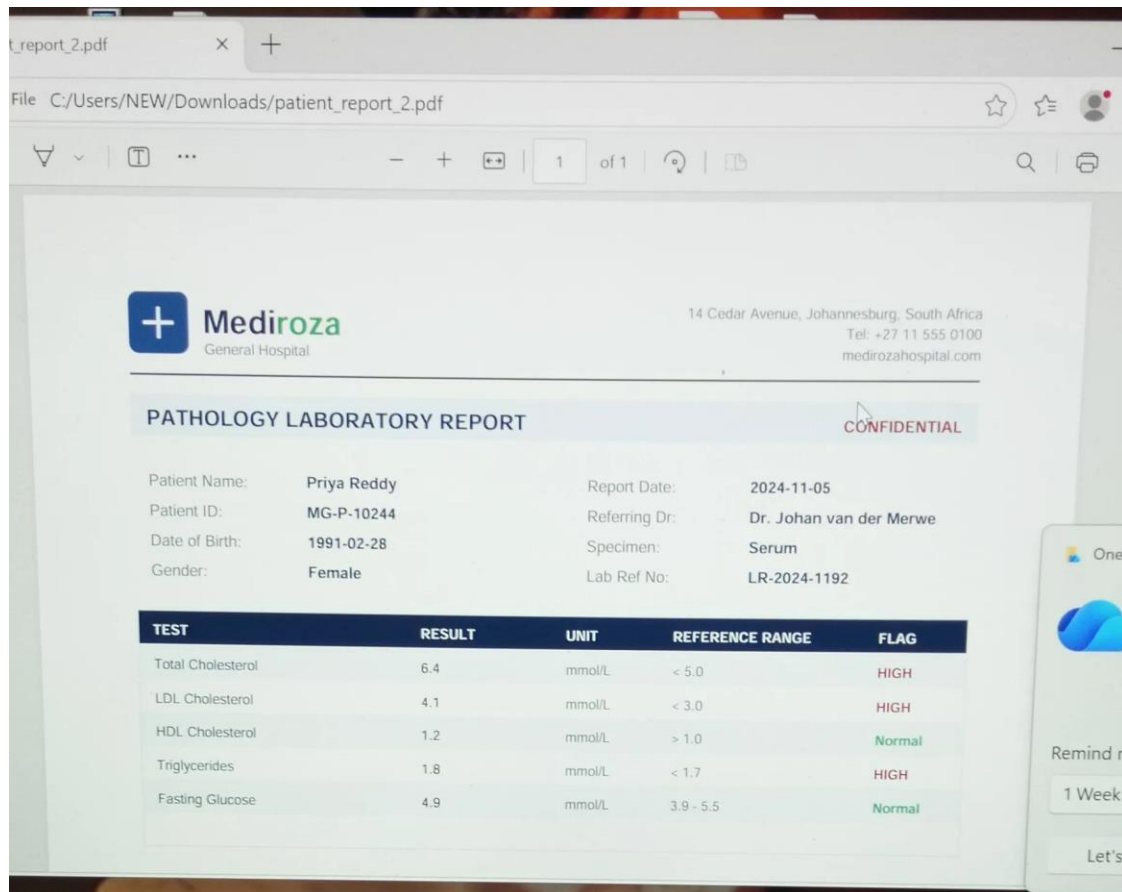


Figure 12: patient_report_2.pdf opened with password password, confirming genuine pathology report content for patient Priya Reddy (LR-2024-1192).

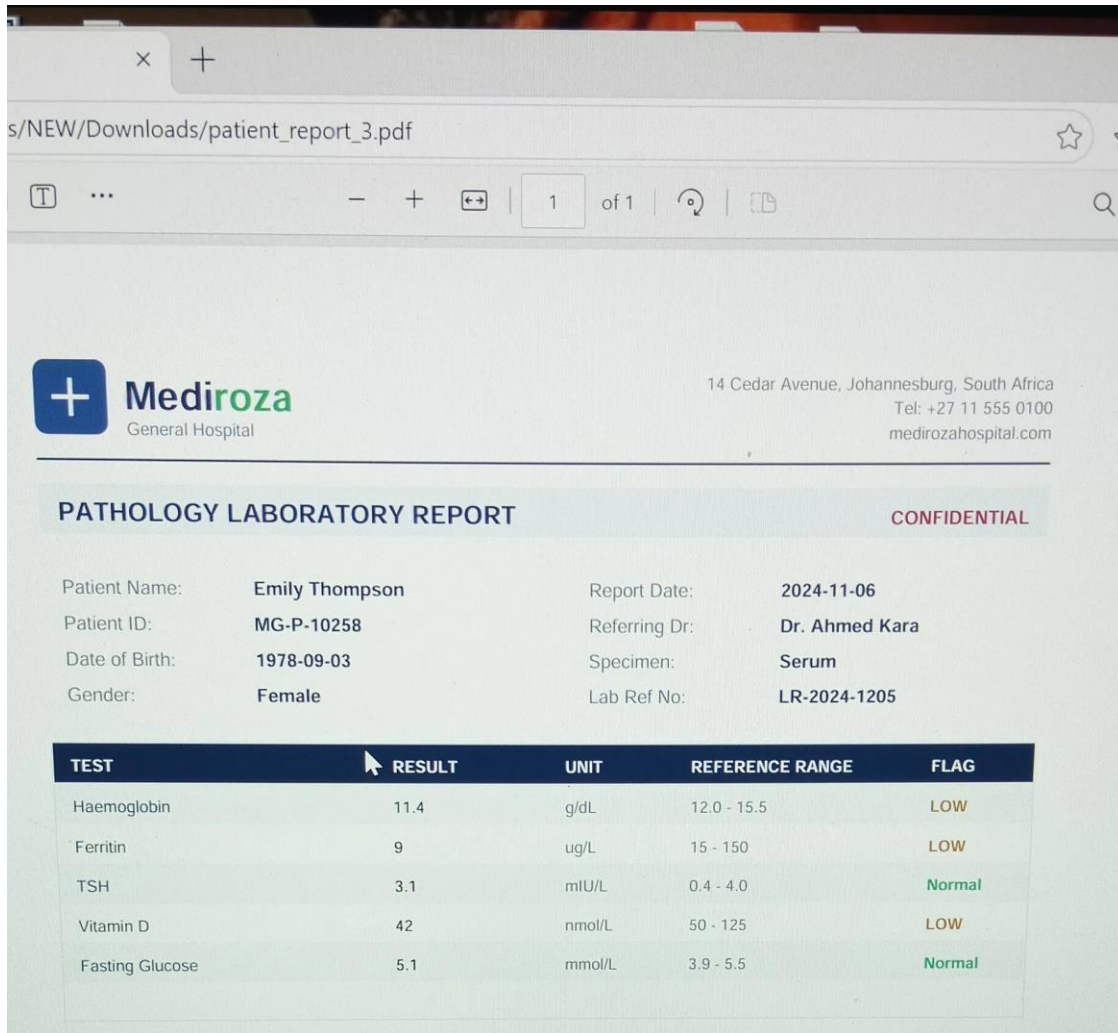


Figure 13: patient_report_3.pdf opened with password !@#\$%^&, confirming genuine pathology report content for patient Emily Thompson (LR-2024-1205).

Impact

Password-protected PDFs are not a substitute for proper access control. Once a file is exfiltrated, as demonstrated in Finding 3, a weak or dictionary-based password (as seen with “123456” and “password”) provides effectively no protection, and even the stronger symbol-based password on patient_report_3.pdf fell within seconds to a modest wordlist. This fully exposes confidential patient lab results for all three patients once the files leave the application.

4. Risk Rating

Rate each vulnerability using the scale below, with justification tied to likelihood and impact.

Rating Scale Reference

Rating	Guidance
Critical	Immediate, severe impact — e.g. full data breach, authentication bypass, remote code execution.
High	Significant impact, may require some conditions to exploit.
Medium	Moderate impact or requires more complex conditions to exploit.
Low	Minor impact, informational, or difficult to exploit.

Findings Summary

Finding	Risk Rating	Justification
1. robots.txt Disclosure	Low	No sensitive data is exposed directly; the file only name the path. However, it materially reduced attacker effort by pointing straight at the two critical findings below, so it is not purely cosmetic.
2. Exposed Database Backup	Critical	Unauthenticated, unauthorized access to a full database backup containing PII (national ID numbers, phone numbers, emails) and payroll data of 30 staff, plus confidential shareholder equity data. Likelihood was high (no barrier beyond knowing the URL) and impact severe (identity theft, financial/reputational harm), meeting the critical bar.
3. SQL Injection Auth Bypass	Critical	A trivial unauthenticated input (admin'--) fully bypass authentication and grants access to confidential patient lab reports. Likelihood is high (no special tooling required) and impact is severe (direct compromised of patient data confidentiality), meeting the critical bar.
4. Weak PDF Password Protection	High	Once a file is exfiltrated, all three PDFs were recoverable offline within seconds

		to a few minutes using a small dictionary. This does not on its own grant initial access (hence not critical), but it removes the last layer of protection on confidential patient data once a breach like finding 3 occurs
--	--	---

5. Recommendations and Remediation

Actionable, specific steps the client should take to fix each identified issue, written for their IT/dev team to act on directly.

Finding	Recommended Remediation
1. robots.txt Disclosure	Do not rely on robots.txt to hide sensitive or administrative directories — it is a public file that only advises search engines, and provides no actual access control. Enforce authentication and server-side authorization on /patient/ and /staff/, and remove /old/ from the web root entirely (see Finding 2).
2. Exposed Database Backup	Remove mediroza_db_backup_2019.sql and any other backup files from web-accessible directories immediately. Store database backups outside the web root, on infrastructure not reachable via HTTP(S). Disable directory listing (e.g. "Options - Indexes") across the entire site. Establish a backup handling policy that treats database dumps as confidential data requiring encryption at rest and restricted, logged access.
3. SQL Injection Auth Bypass	Rewrite all database queries to use parameterized queries / prepared statements — never concatenate user input directly into SQL strings. Use a single, generic error message for failed logins (e.g. "Invalid username or password") to prevent username enumeration. Add rate-limiting or account lockout after repeated failed login attempts. Conduct a full code review of the CMS's authentication module, since this same unsafe pattern likely affects other queries in the application.
4. Weak PDF Password Protection	Move away from relying on password-protected PDFs as the access-control mechanism for confidential patient data. Instead, serve reports only to authenticated, authorized users through the application itself, with server-side access checks and encryption at rest. Where password-protected files remain necessary, enforce strong, randomly generated passwords rather than static or predictable ones.

Overall Recommendation

Findings 2 and 3 (both rated Critical) should be remediated immediately, as they currently allow any unauthenticated user to access confidential patient, staff, and financial data. Findings 1 and 4 should be addressed in the same remediation cycle. NetworkWalks recommends a follow-up retest once fixes are deployed, to confirm each vulnerability has been fully resolved before the application is considered production-ready.